

Documento de entrega

1. Sobre Observables y Asincronía:

a. ¿Por qué las peticiones HTTP en Angular devuelven un Observable en lugar de la data directa? ¿Qué diferencia hay frente a una Promise?

-Las peticiones de HTTP son operaciones asincrónicas, por que la respuesta del servidor no se obtiene de manera inmediata. Es por esta razón que angular devuelve un observable, el cual permite suscribirse a la respuesta cuando esté disponible.

La diferencia de un Observable con un promise, es que el observable ofrece mayor flexibilidad, este puede ser cancelado, encadenar operadores de RxJS como map o catchError, y nos permite manejar los flujos de datos de formas mas compleja. En cambio, una promise representa una única respuesta futura y no ofrece el mismo nivel de control.

b. ¿Qué ocurre si un componente llama a un servicio HTTP, pero nunca hace? subscribe ()? ¿Se ejecuta la petición igualmente?

-Si un componente llama a un servicio HTTP, pero nunca se hace, el servicio devuelve un observable pero la operación queda únicamente definida y no se pone en marcha. Por lo que, el componente no recibe los datos, no se actualiza ninguna variable asociada a la respuesta y tampoco se ejecuta la lógica prevista para manejar un resultado exitoso o un error.

La petición no se ejecuta, en Angular las peticiones realizadas con httpClient solo se envían cuando existe una suscripción Observable. Por lo que, si no se hace. subscribe (), la solicitud de HTTP nunca va a llegar al servidor.

2. Sobre Inyección de Dependencias:

¿Qué ventaja de modularidad da providedIn: 'root' frente a instanciar el servicio manualmente con new CityService() dentro de un componente?

-Un providedIn: 'root' permite que angular cree y gestione una única instancia del servicio para toda la aplicación. Esto hace que haya una estructura más desacoplada, porque los componentes no dependen de como se construye el servicio, sino que simplemente lo solicitan mediante la inyección de dependencias. En cambio, crear el servicio manualmente con new CityService() acopla el componente a la implementación concreta del servicio y dificulta su mantenimiento, reutilización y pruebas.

¿Cómo facilitaría la inyección de dependencias la escritura de pruebas unitarias para estos servicios?

-La inyección de dependencias nos permite facilitar las escrituras de las pruebas unitarias ya que permite reemplazar los servicios reales por versiones simuladas o mocks. Así que, al probar un componente o servicio, no va a ser necesario depender de una API real, una base de datos o una respuesta del backend. Entonces se puede inyectar un servicio falso que devuelva datos controlados y de esta manera verificar de forma precisa cómo responde la lógica de la aplicación. Entonces esto hace que las pruebas sean más rápidas y más confiables, dado que se evalúa únicamente el comportamiento que se quiere comprobar sin depender de factores externos.

3. Sobre Interceptores:

e. ¿Por qué es mejor centralizar el manejo de errores en `HttpInterceptor` en lugar de poner bloques `try/catch` en cada componente?

-Es mejor centralizar el manejo de errores en un `HttpInterceptor` porque este nos permite gestionar de manera uniforme los fallos de las peticiones de HTTP en toda la aplicación. Puesto a que, en lugar de repetir la misma lógica de control de errores en cada componente, el interceptor captura las respuestas fallidas en un único punto y decide como actuar según el origen del error. Asimismo, esta estrategia mejora la organización y el mantenimiento del código. Entonces, si mas adelante se desea cambiar el mensaje mostrado, registrar errores de otra forma o añadir una nueva acción frente a ciertos códigos HTTP, basta con solo modificar el interceptor en un solo lugar. Lo que nos evita duplicación de código, reduce la posibilidad de inconsistencias entre componentes y hace que la aplicación sea más fácil de escalar y mantener.

f. Además del manejo de errores, mencione dos casos de uso adicionales para un interceptor.

1. Agregar encabezados automáticamente a todas las peticiones, por ejemplo, un token de autenticación
2. Registrar o monitorear las peticiones y respuestas, por ejemplo, para medir tiempos de respuesta o hacer logs de auditoría.

4. Sobre el Patrón Maestro-Detalle:

g. ¿Por qué se usa `@Input()` para pasar la ciudad a `CityDetailComponent` en lugar de volver a hacer un GET a `/api/cities/{id}`?

-Se usa `@Input()` porque la información de la ciudad ya fue obtenida previamente en el componente principal al cargar la lista de ciudades. Entonces, al seleccionar una ciudad, ese objeto puede ser enviado directamente al componente de detalle sin la necesidad de realizar una nueva petición al backend para recuperar datos que ya están disponibles. Esto nos permite evitar llamadas HTTP innecesarias, mejora la rapidez de la interfaz y hace que la comunicación entre el componente padre y la del componente hijo sea mucho mas clara. Además, permite que `CityDetailComponent` se concentre en mostrar información complementaria, como el clima y el historial, en lugar de tener que repetir la consulta de la ciudad seleccionada.

h. Observe que en la respuesta de GET `/api/cities`, el campo `country` ya viene como objeto anidado (no solo el `countryId`). ¿Qué ventaja de diseño tiene esta decisión del backend para el frontend?

-Que el campo `country` llegue como objeto completo facilita el trabajo del frontend, porque este permite acceder directamente a los datos como el nombre del país o su código sin tener que hacer una petición adicional por cada ciudad. Entonces, un ejemplo es que la aplicación puede mostrar de inmediato "Bogotá, Colombia" usando `city.country.name`, en

vez de recibir únicamente un identificador y luego tener que consultar de manera separada que país corresponde a ese id. Por lo que, esta decisión simplifica la lógica del cliente, reduce el número de solicitudes para el HTTP, mejora el rendimiento de la aplicación y hace que la información llegue en una estructura más útil para ser presentada directamente en la interfaz.